

Faster Post-Quantum zkSNARK Provers Using the LCH Polynomial Basis

Mohammadtaghi Badakhshan¹, Susanta Samanta² and Guang Gong²

¹ LayerZero Labs mtbadakhshan@gmail.com

² University of Waterloo, Waterloo, Canada [{ssamanta,ggong}@uwaterloo.ca](mailto:ssamanta@uwaterloo.ca)

Abstract. Univariate-polynomial interactive oracle proofs (IOPs) over binary extension fields $\mathbb{F}_{2^m}$ underpin a class of plausibly post-quantum zkSNARKs, but rely heavily on polynomial arithmetic, where large-domain evaluation and division by subspace vanishing polynomials are the dominant prover costs. General-basis additive FFTs, such as Gao–Mateer and Lin–Chung–Han (LCH), accelerate the evaluation but impose a basis-conversion stage costing $O(n(\log n)^2)$ field additions and $O(n \log n)$ field multiplications that dominates in practice. To eliminate basis conversion entirely, we introduce a divide-and-conquer algorithm for polynomial division by vanishing polynomials that operates directly in the LCH polynomial basis, for arbitrary $\mathbb{F}_2$ -basis elements, achieving optimal $O(n \log n)$ complexity. In the LCH polynomial basis, multiplying a vanishing polynomial by the random blinding polynomial reduces to appending random field elements, eliminating the multiplication entirely. We integrate native LCH-basis arithmetic and auxiliary optimizations across all phases of the Aurora IOP. Benchmarks on Preon (a NIST PQC Round-1 signature scheme candidate built on Aurora) show end-to-end signing speedups of $5.0\times$ for Preon-128A and $5.8\times$ for Preon-256C, with the polynomial transform alone $12.6\text{--}17.9\times$ faster.

Keywords: zkSNARK, Post-quantum Digital Signature, Preon, Additive FFT, Binary Extension Fields, CLMUL

1 Introduction

A zero-knowledge succinct non-interactive argument of knowledge (zkSNARK) [BCCT12] lets a prover convince a verifier that it knows a witness for an NP statement, revealing nothing about the witness, with proof size and verification time sublinear in the statement size. Modern constructions compile an *interactive oracle proof* (IOP) [BSCS16] with a *polynomial commitment scheme* (PCS) and apply the Fiat–Shamir [FS87] transform; the PCS choice determines whether the result is plausibly post-quantum secure.

At a high level, IOP designs split into two major paradigms: multivariate sumcheck-based systems such as [Set20, DP25], which avoid polynomial arithmetic in coefficient form entirely, and univariate polynomial IOPs such as [BCR⁺19], which reduce constraint satisfaction to univariate polynomial identities, requiring explicit polynomial arithmetic. This paper focuses on the latter paradigm.

The IOP and PCS also constrain each other in practice. Pairing-based schemes such as KZG [KZG10] are linearly homomorphic but require structured assumptions and are not post-quantum secure. Recent resource estimates indicate that standard elliptic curve cryptography could be more susceptible to quantum attacks than earlier assessments suggested, further motivating the transition to post-quantum alternatives [BZG⁺26]. Hash-based schemes such as FRI [BSBHR18a] are plausibly post-quantum but lack linear homomorphism, forcing the prover to materialize more intermediate polynomials as separate committed oracles; together with the underlying Reed–Solomon (RS) encoding [RS60], this

makes polynomial arithmetic, such as multiplication, division by vanishing polynomials, and large-domain evaluation, all typically realized through *fast Fourier transform* (FFT) algorithms, a dominant component of the prover cost. Hash-based PCSs impose no group-structure requirements on the field, so the same IOP template can be instantiated over prime fields $\mathbb{F}_p$ or binary extension fields $\mathbb{F}_{2^m}$.

Over binary extension fields $\mathbb{F}_{2^m}$, field arithmetic is typically more efficient than in prime fields of comparable bit-length and maps naturally to modern CPU instruction sets. Moreover, symmetric primitives such as AES and SHA-256 are natively expressed in $\mathbb{F}_2$ and related small-field structures, avoiding the bit-decomposition overhead of prime-field arithmetizations [DP25]; recent systems exploit this for efficient zk arguments [FS24] and post-quantum signatures [BdSGK⁺23, CCC⁺23].

Over a prime field $\mathbb{F}_p$, a radix-2 Cooley–Tukey [CT65] multiplicative FFT (or number-theoretic transform) that evaluates a polynomial of degree less than n at n points using butterfly structure, and requires a primitive n th root of unity in $\mathbb{F}_p$, equivalently, $n \mid p - 1$. Over a binary extension field $\mathbb{F}_{2^m}$, one often instead uses an *additive* FFT in which the evaluation domain is an affine $\mathbb{F}_2$ -subspace of $\mathbb{F}_{2^m}$, i.e., a translate of a linear subspace spanned by an $\mathbb{F}_2$ -basis. Main references for additive FFTs include Cantor [Can89], Gao–Mateer [GM10], and Lin, Chung, and Han (LCH) [LCH14].

In the Cantor additive FFT, the evaluation domains are limited to additive affine subspaces determined by the Cantor special basis, whereas the Gao–Mateer and LCH algorithms are applicable to arbitrary choices of basis elements. Badakhshan, Samanta, and Gong [BSG26] achieved a substantial acceleration of the FRI-based Aurora post-quantum zkSNARK by employing the Cantor additive FFT algorithm [Can89]. They further demonstrated, consistent with earlier observations [BC14, CCK⁺17], that using the Cantor special basis significantly improves the runtime of the Gao–Mateer and LCH additive FFT algorithms. Both Gao–Mateer and LCH algorithms include an explicit *basis-conversion* stage, in which polynomials initially represented in the monomial basis are transformed into the Taylor expansion basis in [GM10] or the LCH basis in [LCH14]. The resulting representations are then evaluated using a butterfly structure evaluation stage. The basis-conversion stage accounts for the dominant share of finite-field operations in both Gao–Mateer and LCH FFT algorithms and is significantly optimized when the additive affine evaluation domain is constructed using the Cantor special basis. However, the subsequent evaluation stage, which is comparatively faster, remains independent of the specific choice of basis elements.

In hash-based zkSNARKs, the conventional approach to extending a polynomial from its evaluations on one additive subspace to its evaluations (i.e., its RS encoding) on a larger disjoint evaluation set is to first interpolate the polynomial via an inverse FFT (IFFT) and then evaluate it on the new affine subspace via an FFT. Specifically in binary extension fields, Frigo and Shelat [FS24], show that, when using the LCH additive FFT, this costly basis-conversion step can be avoided, since it suffices to represent the intermediate polynomial in the LCH basis rather than in the monomial basis. While this is sufficient for the IOP underlying Ligerio [AHIV17], the IOPs used in other zkSNARKs, such as Aurora [BCR⁺19], STARK [BSBHR18b], Fractal [COS20], and Polaris [FG22] require further algebraic operations (i.e., polynomial division and multiplication) on the resulting polynomial to derive new polynomials and evaluate them accordingly. This makes the direct use of the LCH basis representation insufficient for these systems, as polynomial multiplication and division require either converting back to the monomial basis or developing equivalent operations natively in the LCH basis. Can polynomial division and multiplication be performed efficiently in the LCH basis without reverting to the monomial representation? This is the central question addressed in this paper.

Contributions

Table 1: End-to-end Preon signature generation time (ms) and speedup over the unmodified reference implementation (*Baseline*). *CLMUL*: carry-less multiply intrinsics for field arithmetic; *NativeLCH*: CLMUL combined with LCH butterfly evaluation, LCH-basis polynomial arithmetic, and auxiliary optimizations.

Configuration	Preon-128A (ms)	Speedup	Preon-256C (ms)	Speedup
Baseline [NIST23]	11,910	1.0×	116,703	1.0×
CLMUL	6,226	1.9×	71,352	1.6×
NativeLCH	2,391	5.0×	20,238	5.8×

1. A divide-and-conquer algorithm for **polynomial division** by subspace vanishing polynomials, presented in both recursive and iterative forms, that operates directly in the LCH polynomial basis for arbitrary $\mathbb{F}_2$ -basis elements. Lin, Al-Naffouri, and Han [LAH16] observed a base-case division property for the LCH alternative (unnormalized) basis; we extend this to a full divide-and-conquer algorithm (Algorithms 1–2) and adapt it to the normalized LCH basis (Algorithm 3), eliminating the $O(n)$ multiplications otherwise needed to convert between the two polynomial bases to ensure seamless integration with the LCH Butterfly. Schoolbook division of a degree- n polynomial by the linearized vanishing polynomial $\mathbb{Z}_m$ requires $O(n \cdot m)$ field multiplications; our algorithm requires $O(n \cdot d)$, where $d = \log_2 n - m - 1$ is typically small in zkSNARK settings, and under the Cantor special basis, the multiplication cost reduces to zero. Additionally, in the LCH basis, **polynomial multiplication** of a vanishing polynomial by the zero-knowledge blinding polynomial reduces to appending random field elements, eliminating the multiplication entirely. Together, these primitives enable the entire prover pipeline to operate natively in the LCH basis, removing the basis-conversion stage (costing $O(n(\log n)^2)$ field additions and $O(n \log n)$ field multiplications) from every additive FFT and IFFT call and reducing each transform to the $O(n \log n)$ butterfly evaluation alone.
2. An optimized implementation of the Aurora zkSNARK [BCR⁺19], evaluated on the Preon [CCC⁺23] post-quantum signature scheme, a first-round candidate in NIST’s additional PQC signature standardization process [NIST23], whose prover is built on the Aurora IOP. The algorithmic contributions above allow us to replace the Gao–Mateer FFT with the more efficient LCH butterfly evaluation. Our further modifications include sparse encoding of R1CS matrix–vector products and carry-less multiplication (CLMUL) intrinsics for field arithmetic. All changes are confined to the arithmetic layer and do not alter the proof transcript, preserving the security guarantees of the underlying protocol. Table 1 summarizes the end-to-end impact on the Aurora prover cost for Preon-128A (NIST Level I, aggressive security assumptions) and Preon-256C (NIST Level V, conservative security assumptions), reporting CLMUL separately so that the algorithmic speedup can be assessed independently. Since these optimizations target the polynomial arithmetic layer of the Aurora IOP, they are directly applicable to any univariate-polynomial IOP prover over binary extension fields, including STARK [BSBHR18b], Fractal [COS20], and Polaris [FG22].

2 Related Works

2.1 Additive FFT Algorithm

The analysis in [BSG26] shows that the Gao–Mateer [GM10] and LCH [LCH14] algorithms both consist of a basis-conversion stage followed by a butterfly evaluation stage. The butterfly evaluation costs $n \log_2 n$ additions and $\frac{1}{2}n \log_2 n$ multiplications for both algo-

rithms. Over a general basis, Gao–Mateer basis conversion requires $\frac{1}{4}n(\log_2 n)^2 - \frac{1}{4}n \log_2 n$ additions and $n \log_2 n - n + 1$ multiplications, while LCH requires $O(n(\log_2 n)^2)$ additions and $O(n \log_2 n)$ multiplications. Switching to the Cantor special basis eliminates all multiplications in the basis-conversion stage of both algorithms; the Gao–Mateer addition count remains $\frac{1}{4}n(\log_2 n)^2 - \frac{1}{4}n \log_2 n$, whereas the LCH addition count improves to $O(n \log_2 n \log_2 \log_2 n)$. The Cantor FFT, by contrast, has no basis-conversion stage and does not use a butterfly structure; it operates through a recursive polynomial-division procedure requiring $\frac{1}{2}n \log_2 n$ multiplications and $\frac{1}{2}n \log_2 n + \frac{1}{2}n \sum_{r=0}^{\log_2 n-1} 2^{\text{wt}(r)}$ additions in total [BSG26]. In end-to-end Aurora prover benchmarks for circuit sizes 2^{14} through 2^{19} , replacing the Gao–Mateer FFT with the Cantor FFT yielded speedups of up to $\sim 1.75\times$, growing with circuit size as the basis-conversion cost becomes increasingly dominant. In the present work, we use only the butterfly evaluation stage of the LCH FFT, and evaluate our optimizations on the Preon [CCC⁺23] post-quantum signature scheme, which is built on the Aurora proving system.

2.2 Polynomial Division

Univariate-polynomial-based IOPs reduce constraint satisfaction to polynomial divisibility, as if a constraint polynomial $C(X)$ encodes the prover’s computation trace over an evaluation domain H , then the statement “all constraints are satisfied” is equivalent to $C(\alpha) = 0$ for every $\alpha \in H$, which holds if and only if the vanishing polynomial $Z_H(X) = \prod_{\alpha \in H} (X - \alpha)$ divides $C(X)$. The prover exhibits a low-degree quotient $Q(X) = C(X)/Z_H(X)$, and the verifier checks the identity $C(\zeta) = Q(\zeta) \cdot Z_H(\zeta)$ at a random challenge ζ ; soundness follows from the Schwartz–Zippel lemma [Sch80, Zip79]. Computing this quotient is a core arithmetic primitive in every univariate-polynomial-based zkSNARK prover [BCCT12, Gro16, GWC19, BCR⁺19, COS20].

Prime-field approaches. Over a prime field $\mathbb{F}_p$ with $n \mid p - 1$, the evaluation domain is a multiplicative subgroup $H = \{1, \omega, \dots, \omega^{n-1}\}$ with vanishing polynomial $Z_H(X) = X^n - 1$. A natural approach to computing the quotient $Q(X) = C(X)/Z_H(X)$ is to evaluate both polynomials on H and divide pointwise; however, $Z_H(\omega^i) = 0$ for every $\omega^i \in H$, making the pointwise quotient an indeterminate $0/0$ form. Groth16 implementations (libsnark [BSCG⁺], bellman [BGoc], arkworks [ark]) circumvent this by evaluating on a multiplicative coset $gH = \{g, g\omega, \dots, g\omega^{n-1}\}$ with $g \notin H$, where Z_H takes the nonzero constant value $g^n - 1$ at every point, reducing division to scaling all n evaluations by the single precomputed inverse $(g^n - 1)^{-1}$. This requires $2n$ -th roots of unity and hence halves the maximum circuit size for a given field. PLONK-family systems [GWC19] (halo2 [BLG⁺], gnark [BGPoc], Plonky2 [BWL⁺oc]) combine gate, permutation, and lookup constraints into a numerator of degree up to kn , so the quotient has degree up to $(k - 1)n$ and can no longer be captured by n evaluation points on a single coset. These systems therefore work on an extended coset gH' of size kn , where the shift $g \notin H'$ ensures that Z_H is nonzero at every evaluation point (since $H \subset H'$, evaluating on H' itself would encounter zeros of Z_H); on this coset Z_H takes only k distinct nonzero values, and a precomputed table of their batch-inverted values [Mon87] reduces the per-proof division to $O(kn)$ pointwise multiplications. Chalkias et al. [CJL⁺24] propose a unified approach that resolves the $0/0$ indeterminacy directly via L’Hôpital’s rule for formal derivatives, enabling quotient computation on H without a coset shift. This eliminates the need for $2n$ -th roots of unity and doubles the maximum circuit size. Orthogonally, snarkjs [BPoc] performs division in the monomial coefficient basis, where the sparsity of the denominator $X^n - 1$ (only two nonzero terms) reduces the schoolbook algorithm to $O(kn)$ additions for a dividend of degree kn .

Binary-field division. Over $\mathbb{F}_{2^k}$ the evaluation domain is an $\mathbb{F}_2$ -linear subspace $W \subseteq \mathbb{F}_{2^k}$ of dimension m , whose vanishing polynomial $\mathbb{Z}_m(X) = \prod_{a \in W} (X + a)$ is $\mathbb{F}_2$ -linearized with only $m + 1$ nonzero terms (Section 3). Since $\mathbb{Z}_m$ is monic, schoolbook long division touches only m non-leading coefficients per step, requiring $(L - 2^m) \cdot m$ field multiplications and as many additions for a dividend of length L , a factor of $2^m/m$ fewer than for a dense divisor of the same degree. Under the Cantor special basis [Can89] all coefficients of $\mathbb{Z}_m(X)$ lie in $\mathbb{F}_2$, with exactly $2^{\text{wt}(m)}$ nonzero terms, where $\text{wt}(m)$ denotes the Hamming weight of m [BSG26]. Schoolbook division of a polynomial of length L by such a $\mathbb{Z}_m$ requires no field multiplications, and only $(L - 2^m) \cdot (2^{\text{wt}(m)} - 1)$ additions.

Similar to prime-field coset approaches, division can also proceed in the evaluation domain: an additive inverse FFT recovers the numerator in coefficient form, a forward additive FFT evaluates the numerator on an additive coset $v + W$ with $v \notin W$, where $\mathbb{Z}_m(v + x) = \mathbb{Z}_m(v)$ for all $x \in W$ by the linearized property, reducing division single scalar multiplications by $\mathbb{Z}_m(v)^{-1}$. This requires the numerator to have degree less than 2^{m+1} , so that the quotient has degree $< 2^m$ and is uniquely determined by 2^m evaluation points. When the quotient degree exceeds 2^m and evaluation on an extended coset $v + W'$ with $W' \supset W$ is needed, $\mathbb{Z}_m$ is no longer constant but takes only $|W'|/|W|$ distinct nonzero values on the coset, and a precomputed table of their inverses reduces division to $|W'|$ pointwise multiplications.

In FRI-based zkSNARKs [BSBHR18a, BCR⁺19, BSBHR18b], the prover's constraint polynomials are initially constructed as evaluations on the constraint domain, and each committed polynomial must subsequently be evaluated on a codeword domain whose size exceeds the polynomial degree by a factor of ρ^{-1} (the inverse RS rate). Therefore, regardless of the division algorithm used, whether pointwise on a coset or via a coefficient-space recurrence, the quotient polynomial must ultimately be recovered in coefficient form so that an FFT can evaluate it over the codeword domain. Consequently, a fast division algorithm operating directly in the coefficient representation is particularly attractive, as it avoids the additional inverse-and-forward transform pair that evaluation-domain approaches require to mitigate the 0/0 indeterminacy on the constraint domain.

3 Preliminaries

In this section, we recall the basic definitions and algebraic notions used throughout the paper. These concepts provide the foundation for the constructions and analysis developed in the subsequent sections.

3.1 Algebraic Foundations

Finite Field Let $\mathbb{F}_{2^k}$ be a degree- k extension field of $\mathbb{F}_2$, and let $1_{\mathbb{F}_{2^k}}$ denote the multiplicative identity. Fix an $\mathbb{F}_2$ -basis $\{\beta_0, \beta_1, \dots, \beta_{k-1}\}$ of $\mathbb{F}_{2^k}$. Thus, the elements $\beta_0, \beta_1, \dots, \beta_{k-1}$ are linearly independent over $\mathbb{F}_2$, and every element of $\mathbb{F}_{2^k}$ admits a unique representation as an $\mathbb{F}_2$ -linear combination of these basis elements. For each integer $0 \leq i < 2^k$, let $i = \sum_{j=0}^{k-1} i_j 2^j$, with $i_j \in \{0, 1\}$, be its binary expansion. We enumerate the elements of $\mathbb{F}_{2^k}$ as $\{\eta_0, \eta_1, \dots, \eta_{2^k-1}\}$, where $\eta_i = \sum_{j=0}^{k-1} i_j \beta_j$.

Univariate Polynomial Let $\mathbb{F}_{2^k}[x]/(x^{2^k} - x)$ denote the polynomial ring over $\mathbb{F}_{2^k}$ in the indeterminate x . A univariate polynomial of degree less than n is denoted as

$$f(x) = \sum_{i=0}^{n-1} f_i x^i \in \mathbb{F}_{2^k}[x]/(x^{2^k} - x), \quad f_i \in \mathbb{F}_{2^k}.$$

Subspace Vanishing Polynomial For $m < k$, let $W_m = \langle \beta_0, \beta_1, \dots, \beta_{m-1} \rangle$ denote an m -dimensional $\mathbb{F}_2$ -linear subspace of $\mathbb{F}_{2^k}$. The *vanishing polynomial* of the subspace W_m is defined by $\mathbb{Z}_m(x) = \prod_{i=0}^{2^m-1} (x + \eta_i)$. Thus, $\deg(\mathbb{Z}_m) = 2^m$. Moreover, $\mathbb{Z}_m(x)$ is an $\mathbb{F}_2$ -linearized polynomial and hence satisfies

$$\mathbb{Z}_m(x + y) = \mathbb{Z}_m(x) + \mathbb{Z}_m(y), \forall x, y \in \mathbb{F}_{2^k}$$

Since $W_{m+1} = W_m \cup (\beta_m + W_m)$, the vanishing polynomials satisfy the recurrence

$$\mathbb{Z}_{m+1}(x) = (\mathbb{Z}_m(x))^2 - \mathbb{Z}_m(\beta_m) \mathbb{Z}_m(x).$$

3.2 LCH Polynomial Basis

The LCH polynomial basis [LCH14] for $\mathbb{F}_{2^k}[x]$ is constructed from subspace vanishing polynomials, and is defined as

$$\bar{\mathbb{X}} = \{\bar{X}_0(x), \bar{X}_1(x), \dots, \bar{X}_{2^k-1}(x)\}, \quad \bar{X}_i(x) = \frac{X_i(x)}{p_i},$$

where $\deg(\bar{X}_i(x)) = i$. Here,

$$X_i(x) := \prod_{j=0}^{k-1} (\mathbb{Z}_j(x))^{i_j}, \quad p_i := \prod_{j=0}^{k-1} (\mathbb{Z}_j(\beta_j))^{i_j}, \quad (1)$$

and $(i_0, i_1, \dots, i_{k-1}) \in \{0, 1\}^k$ denotes the binary representation of i .

A univariate polynomial $f(x) \in \mathbb{F}_{2^k}[x]/(x^{2^k} - x)$ of degree less than n can be expressed in the LCH basis as $f(x) = \sum_{i=0}^{n-1} f_i^{\bar{\mathbb{X}}} \bar{X}_i(x)$. The *LCH basis-conversion* algorithms [LAHC16] convert the monomial-basis coefficient vector $(f_0, f_1, \dots, f_{n-1})$ of a polynomial to its LCH-basis coefficient vector $(f_0^{\bar{\mathbb{X}}}, f_1^{\bar{\mathbb{X}}}, \dots, f_{n-1}^{\bar{\mathbb{X}}})$.

The *LCH alternative polynomial basis* [LAH16] is defined as

$$\mathbb{X} = \{X_0(x), X_1(x), \dots, X_{2^k-1}(x)\}. \quad (2)$$

The *LCH Butterfly* [LCH14, LAH16] is the evaluation stage of the LCH additive FFT: given the LCH-basis coefficient vector $(f_0^{\bar{\mathbb{X}}}, \dots, f_{n-1}^{\bar{\mathbb{X}}})$ of a polynomial $f(x)$ and an affine coset $v + W$ with $W = \langle \beta_0, \dots, \beta_{k-1} \rangle$, it computes the evaluation vector $(f(\alpha))_{\alpha \in v+W}$ via a Cooley–Tukey-style butterfly network in $n \log_2 n$ additions and $\frac{1}{2}n \log_2 n$ multiplications. The *LCH IButterfly* is its inverse: given evaluations $(f(\alpha))_{\alpha \in v+W}$, it recovers the LCH-basis coefficients $(f_0^{\bar{\mathbb{X}}}, \dots, f_{n-1}^{\bar{\mathbb{X}}})$ with the same operation count.

3.3 Aurora IOP

Aurora [BCR⁺19] is a univariate-polynomial IOP for R1CS satisfiability over $\mathbb{F}_{2^k}$. An R1CS instance consists of three sparse matrices $A, B, C \in \mathbb{F}_{2^k}^{m \times (n+1)}$; a prover holding witness $w \in \mathbb{F}_{2^k}^{n-k}$ demonstrates that the full assignment $z = (1, v, w)$ satisfies $Az \circ Bz = Cz$ (Hadamard product). The constraint domain H_1 and variable domain H_2 are $\mathbb{F}_2$ -linear subspaces of $\mathbb{F}_{2^k}$ with $|H_1| = m$ and $|H_2| = n + 1$; the codeword domain L is a disjoint subspace with $|L| \geq 2t/\rho$, where $t = \max(m, n+1)$ and ρ is the Reed–Solomon [RS60] rate. Every polynomial oracle is materialized as evaluations on L and committed via a Merkle tree; verifier challenges are derived by Fiat–Shamir hashing (BCS transformation [BSCS16]).

The prover pipeline (Figure 2) proceeds through five IOP phases. We describe each phase in terms of the polynomial primitives it requires and their costs.

Phase 1 (Oracle construction). The prover encodes the matrix-vector products Az, Bz, Cz as low-degree polynomials over H_1 via IFFT and evaluates each on L via FFT. The witness oracle f_w requires additional work: defining $\hat{f}'_w(x) = \hat{f}_z(x) - \hat{f}_{(1,v)}(x)$ on the variable domain H_2 , the polynomial $\hat{f}'_w$ vanishes on the public-input subdomain $H_2^{\leq k}$, so recovering $\hat{f}_w = \hat{f}'_w / \mathbb{Z}_{H_2^{\leq k}}$ is an *exact polynomial division by a vanishing polynomial* in coefficient form, after which $\hat{f}_w$ is evaluated on L via FFT. For zero knowledge, each oracle polynomial is blinded by adding $\mathbb{Z}_H \cdot h$ for a random h , which is a *polynomial multiplication by a vanishing polynomial*. The four codewords $f_w, f_{Az}, f_{Bz}, f_{Cz}$ are committed in BCS Round 0 together with masking oracles for the sumcheck (Phase 3) and FRI (Phase 5). The virtual oracle $f_z = f_w \cdot \mathbb{Z}_{H_2^{\leq k}} + f_{(1,v)}$ is then assembled on L (Phase 1' in the figure).

Phase 2 (Lincheck). A verifier challenge α defines interpolation polynomials $\hat{p}_\alpha$ and $\hat{p}_\alpha^{(M)}$ for $M \in \{A, B, C\}$. $\hat{p}_\alpha$ interpolates the powers $(\alpha^0, \dots, \alpha^{m-1})$ on H_1 ; each $\hat{p}_\alpha^{(M)}$ interpolates $M^\top \vec{\alpha}$ on H_2 , requiring one sparse matrix-vector product per matrix. All four polynomials are recovered via IFFT on the summation domain $H_1 \cup H_2$ (size t) and evaluated on L via FFT. The three lincheck claims $q_M = \hat{f}_{Mz} \cdot \hat{p}_\alpha - \hat{f}_z \cdot \hat{p}_\alpha^{(M)}$ are combined with random weights and evaluated on L to form a single virtual oracle.

Phase 3 (Sumcheck). The three lincheck claims $\sum_{a \in H_1 \cup H_2} \hat{q}_M(a) = 0$ are amortized with random weights s_1, s_2, s_3 into a single univariate sumcheck on $\hat{f} = \sum_j s_j \hat{q}_j + \hat{m}$, where $\hat{m}$ is a masking polynomial for zero knowledge (defined below). The prover decomposes

$$\hat{f}(X) = \hat{g}(X) + \beta X^{t-1} + \mathbb{Z}_{H_1 \cup H_2}(X) \cdot \hat{h}(X) \quad (3)$$

Since $\deg(\hat{f}) < 2t$, the prover evaluates $\hat{f}$ on a minimal subspace of size $2t$ and applies an IFFT of that size to recover its coefficients. *Polynomial division with remainder* by $\mathbb{Z}_{H_1 \cup H_2}$ is then performed in coefficient form. Both $\hat{h}$ and $\hat{g}$ are then evaluated on L via forward FFTs; the quotient evaluation $h = \hat{h}|_L$ is committed in BCS Round 1, while $\hat{g}|_L$ forms a virtual oracle for the low-degree test (Phase 3'). For zero knowledge, a masking polynomial $\hat{m} = \mathbb{Z}_H \cdot h + g$ is constructed, where h is random and g must satisfy $\sum_{a \in H} g(a) = 0$. Computing $\mathbb{Z}_H \cdot h$ requires a *polynomial multiplication by a vanishing polynomial*, and enforcing $\sum_{a \in H} g(a) = 0$ requires characterizing which polynomials sum to zero over H . The masking oracle $m|_L$ is committed in BCS Round 0.

Phase 4 (Rowcheck). The Hadamard constraint is verified by computing $(f_{Az}(a) \cdot f_{Bz}(a) - f_{Cz}(a)) / \mathbb{Z}_{H_1}(a)$ for each $a \in L$. Since L is disjoint from H_1 , $\mathbb{Z}_{H_1}$ is nonzero on L and the numerator vanishes on H_1 by the R1CS constraint, so this yields a well-defined low-degree polynomial.

Phase 5 (Low-degree test). All oracle and virtual-oracle codewords are combined via a random linear combination into a single codeword, which is tested for proximity to a Reed–Solomon code using the FRI protocol [BSBHR18a]. For zero knowledge, a random blinding polynomial (committed in BCS Round 0) is included in the linear combination.

3.4 Preon Signature Scheme

Preon [CCC⁺23] is a post-quantum digital signature scheme constructed from Aurora. The underlying NP statement is AES evaluation, where the signer proves knowledge of a secret key sk such that $\text{AES}_{sk}(P_i) = C_i$ for public plaintext–ciphertext pairs (P_i, C_i) . The AES circuit (e.g., key schedule, SubBytes, MixColumns, and their inverses) is encoded as a fixed R1CS instance over $\mathbb{F}_{2^k}$ with $\{(P_i, C_i)\}$ as public input and sk together with all intermediate AES state values as private witness.

Key generation. Sample a random AES key sk and random blocks P_i , and set $pk = \{(P_i, \text{AES}_{sk}(P_i))\}$.

Signing. Compute the full R1CS assignment $z = (1, pk, sk, \text{AES trace})$ and run the Aurora prover. The Fiat–Shamir transcript is initialized with a hash of the message m , binding the proof to m . The signature is the serialized BCS proof.

Verification. Reconstruct the R1CS instance from pk and run the Aurora verifier on the proof, without access to sk .

4 Polynomial Division Algorithm

In the Aurora prover pipeline (Section 3.3), the LCH IButterfly recovers LCH-basis coefficients from evaluations, and the LCH Butterfly evaluates from LCH-basis coefficients. If polynomial division can be performed directly in the LCH basis, the entire pipeline, i.e., interpolation, division, and re-evaluation, operates without ever converting to or from the monomial representation, eliminating the heavy basis-conversion stage from every transform. This is the goal of the algorithms developed in this section.

Given univariate polynomials $A(x)$ and $B(x)$ with $\deg(B) < \deg(A)$, the polynomial division algorithm finds a unique quotient $Q(x)$ and remainder $R(x)$ such that $A(x) = B(x)Q(x) + R(x)$, where $\deg(R) < \deg(B)$. In this section, we present an efficient divide-and-conquer algorithm for division by the subspace vanishing polynomial $\mathbb{Z}_m(x)$, first in the LCH alternative basis and then in the LCH basis. The latter is particularly important in practice, as the LCH IButterfly produces and the LCH Butterfly consumes normalized LCH-basis coefficients, performing division directly in the LCH basis avoids the $O(n)$ -multiplication cost of converting to the alternative basis and back.

We first recall a fundamental property of the LCH basis that characterizes the quotient when the dividend degree is strictly bounded. Our algorithm relies on the following structural property of the LCH alternative basis.

Lemma 1. [LAH16] *Let $m \geq 0$, and let $A(x)$ be a polynomial of degree $\deg(A) < 2^{m+1}$, expressed in the LCH alternative basis as $A(x) = \sum_{j=0}^{h-1} a_j^{\mathbb{X}} X_j(x)$, where $h \leq 2^{m+1}$. The quotient $Q(x)$ obtained by dividing $A(x)$ by $\mathbb{Z}_m(x)$ is given by $Q(x) = \sum_{\ell=0}^{2^m-1} a_{\ell+2^m}^{\mathbb{X}} X_{\ell}(x)$.*

To see why this holds, note that we can split the sum in $A(x)$ at 2^m and factor out $\mathbb{Z}_m(x)$ from the upper terms to write

$$A(x) = \sum_{i=0}^{2^m-1} a_i^{\mathbb{X}} X_i(x) + \sum_{i=2^m}^{h-1} a_i^{\mathbb{X}} X_i(x) = \sum_{i=0}^{2^m-1} a_i^{\mathbb{X}} X_i(x) + \mathbb{Z}_m(x) \sum_{i=2^m}^{h-1} a_i^{\mathbb{X}} X_{i-2^m}(x).$$

It is worth mentioning that, although the factorization $X_{2^m+k}(x) = X_{2^m}(x) \cdot X_k(x)$ is valid, it only applies when $k < 2^m$. As the polynomial degree scales beyond the 2^{m+1} threshold, the basis elements rely on overlapping subspace vanishing polynomials defined by the nonlinear recurrence $\mathbb{Z}_{m+1}(x) = (\mathbb{Z}_m(x))^2 - \mathbb{Z}_m(\beta_m)\mathbb{Z}_m(x)$. Because of this recurrence, direct factorization fails for higher degrees. For instance, dividing a high-degree basis element such as $X_{2^m+d}(x)$ by $\mathbb{Z}_m(x)$ does not yield a single quotient term. Instead, it yields 2^d -term expansion polynomial defined as $D_{d,m}(x) = \sum_{i=0}^{2^d-1} p_{2^m(2^d-i-1)} X_{2^m i}(x)$. A sequential approach would process such high-degree basis terms one at a time. Since each division by $\mathbb{Z}_m(x)$ produces a multi-term expansion over lower basis elements, the same lower coefficients would be updated repeatedly. Algorithm 1 avoids this by recursively splitting the polynomial and applying the expansion only once per block during the merge step through the term $R_1(x) \left(\sum_{i=0}^{2^d-1} p_{2^m(2^d-i-1)} X_{2^m i}(x) \right)$.

Algorithm 1 Recursive polynomial division by $\mathbb{Z}_m(x)$ in the LCH alternative basis

Require: $A(x) = \sum_{i=0}^{2^{d+m+1}-1} a_i^{\mathbb{X}} X_i(x) \in \mathbb{F}_{2^k}[x]/(x^{2^k} - x)$ of degree $< 2^{d+m+1}$ with $d + m + 1 \leq k$, expressed in the LCH alternative basis.

Ensure: $Q(x), R(x) \leftarrow \text{RecDivLCHAlt}(A(x), d, m)$: $Q(x)$ of degree $< 2^{d+m+1} - 2^m$, and $R(x)$ of degree $< 2^m$ are in the LCH alternative basis and $A(x) = \mathbb{Z}_m(x) \cdot Q(x) + R(x)$.

1: **if** $d = 0$ **then**

2: Let

$$R(x) = \sum_{i=0}^{2^m-1} a_i^{\mathbb{X}} X_i(x) \quad \text{and} \quad Q(x) = \sum_{i=0}^{2^m-1} a_{i+2^m}^{\mathbb{X}} X_i(x).$$

3: **return** $Q(x), R(x)$

4: **end if**

5: Let

$$A_0(x) = \sum_{i=0}^{2^{m+d}-1} a_i^{\mathbb{X}} X_i(x) \quad \text{and} \quad A_1(x) = \sum_{i=0}^{2^{m+d}-1} a_{i+2^{m+d}}^{\mathbb{X}} X_i(x).$$

6: Compute

$$\begin{aligned} Q_0(x), R_0(x) &\leftarrow \text{RecDivLCHAlt}(A_0(x), d-1, m), \\ Q_1(x), R_1(x) &\leftarrow \text{RecDivLCHAlt}(A_1(x), d-1, m). \end{aligned}$$

7: Compute

$$\begin{aligned} R(x) &= R_0(x), \\ Q(x) &= Q_0(x) + R_1(x) \times \left(\sum_{i=0}^{2^d-1} p_{2^m \cdot (2^d-i-1)} \cdot X_{2^m \cdot i}(x) \right) + Q_1(x) \times X_{2^{d+m}}(x). \end{aligned}$$

8: **return** $Q(x), R(x)$

The following theorem establishes the correctness of Algorithm 1. Its proof is deferred to Appendix A.1.

Theorem 1. *Algorithm 1 correctly computes the quotient $Q(x)$ and remainder $R(x)$ of the division of $A(x)$ by the vanishing polynomial $\mathbb{Z}_m(x)$.*

The following lemma establishes the equivalence between Algorithm 2 and Algorithm 1. Its proof is deferred to Appendix A.2.

Lemma 2. *For every valid input polynomial $A(x) = \sum_{i=0}^{2^{d+m+1}-1} a_i^{\mathbb{X}} X_i(x)$, Algorithm 2 returns the same pair $(Q(x), R(x))$ as Algorithm 1. Hence, Algorithm 2 is an iterative realization of Algorithm 1.*

4.1 Detailed Cost Analysis of Algorithm 2

Algorithm 2 represents the explicit iterative unrolling of the recursive bisection defined in Algorithm 1. To quantify the exact arithmetic cost of the polynomial division, we analyze the finite field operations. Let the input polynomial $A(x)$ possess a degree bounded by $n = 2^{m+d+1}$, which is processed as an array of 2^{d+1} distinct blocks, each containing exactly 2^m field elements.

Proposition 1. *Algorithm 2 performs exactly $2^m(d \cdot 2^d - 2^d + 1)$ field multiplications and the same number of field additions.*

Proof. The computational cost is entirely determined by the innermost operation in line 8, which performs one field multiplication and one field addition per execution:

$$\bar{\mathbf{q}}[w \cdot 2^m + i] \leftarrow \bar{\mathbf{q}}[w \cdot 2^m + i] + \bar{\mathbf{q}}[t \cdot 2^m + i] \cdot \gamma.$$

The scalar $\gamma \leftarrow p_{2^m(2^s-u-1)}$ is fetched once per (s, u) -pair at line 4 and reused across all v and i iterations, contributing no additional multiplications.

Algorithm 2 Polynomial division by $\mathbb{Z}_m(x)$ in the LCH alternative basis

Require: $A(x) = \sum_{i=0}^{2^{d+m+1}-1} a_i^{\mathbb{X}} X_i(x) \in \mathbb{F}_{2^k}[x]/(x^{2^k} - x)$ of degree $< 2^{d+m+1}$ with $d + m + 1 \leq k$, expressed in the LCH alternative basis.

Ensure: $Q(x), R(x) \leftarrow \text{DivLCHAlt}(A(x), d, m)$: $Q(x)$ of degree $< 2^{d+m+1} - 2^m$, and $R(x)$ of degree $< 2^m$ are in the LCH alternative basis and $A(x) = \mathbb{Z}_m(x) \cdot Q(x) + R(x)$.

- 1: $\vec{r} := [a_i^{\mathbb{X}} \mid 0 \leq i < 2^m]$, and $\vec{q} := [a_i^{\mathbb{X}} \mid 2^m \leq i < 2^{m+d+1}]$.
- 2: **for** $s = 1, \dots, d$ **do**
- 3: **for** $u = 0, 1, \dots, 2^s - 2$ **do**
- 4: $\gamma \leftarrow p_{2^m \cdot (2^s - u - 1)}$
- 5: **for** $v = 0, 1, \dots, 2^{d-s} - 1$ **do**
- 6: $t \leftarrow v \cdot 2^{s+1} + 2^s - 1, \quad w \leftarrow v \cdot 2^{s+1} + u$
- 7: **for** $i = 0, \dots, 2^m - 1$ **do**
- 8: $\vec{q}[w \cdot 2^m + i] \leftarrow \vec{q}[w \cdot 2^m + i] + \vec{q}[t \cdot 2^m + i] \cdot \gamma$
- 9: **end for**
- 10: **end for**
- 11: **end for**
- 12: **end for**
- 13: Compute $Q(x), R(x)$ from $\vec{q}$ and $\vec{r}$
- 14: **return** $Q(x), R(x)$

The total execution count of line 8 follows directly from the four nested loops. At each level $s \in \{1, \dots, d\}$, the second loop (line 3) runs $2^s - 1$ times, the third loop (line 5) runs 2^{d-s} times, and the inner loop (line 7) runs 2^m times, giving a contribution of $(2^s - 1) \cdot 2^{d-s} \cdot 2^m$ field multiplications at level s . Summing over $s = 1, \dots, d$, we have the exact multiplication cost of the algorithm as

$$\begin{aligned} \sum_{s=1}^d (2^s - 1) \cdot 2^{d-s} \cdot 2^m &= 2^m \sum_{s=1}^d (2^d - 2^{d-s}) \\ &= 2^m (d \cdot 2^d - 2^d + 1). \end{aligned}$$

Since each execution of line 8 contributes exactly one multiplication and one addition, both totals equal to $2^m(d \cdot 2^d - 2^d + 1)$. $\square$

Corollary 1. *If the evaluation domain is constructed using the Cantor special basis (i.e., $\beta_0 = 1$ and $\beta_i^2 + \beta_i = \beta_{i-1}$ for $1 \leq i \leq k - 1$), the scalar constants satisfy $p_i = 1$ for all i (see [GM10, Lemma 1]). Consequently, the scaling factor becomes universally $\gamma = 1$. This eliminates the need for multiplication entirely, reducing the multiplication cost of Algorithm 2 to exactly 0 while preserving the required $2^m(d \cdot 2^d - 2^d + 1)$ field additions.*

Remark 1. In terms of the total input size $n = 2^{d+m+1}$, substituting $d = \log_2 n - (m + 1)$ into the exact operation count yields

$$2^m(d \cdot 2^d - 2^d + 1) = \frac{n}{2} (\log_2 n - (m + 2)) + 2^m = O(n \log_2 n).$$

Thus, Algorithm 2 perfectly achieves quasi-linear complexity in the input size n .

While Algorithms 1 and 2 operate in the LCH alternative basis, the LCH Butterfly and IButterfly produce and consume coefficients in the normalized LCH basis. Using the alternative-basis algorithms would therefore require $O(n)$ multiplications (scaling each coefficient by p_i or p_i^{-1}) to convert before and after each division step. Algorithm 3 not only eliminates the basis conversion overhead but also reduces the total multiplication count. Figure 1 contrasts the data flow of both algorithms for $d = 3$.

The following theorem establishes the correctness of Algorithm 3. Its proof is deferred to Appendix A.3.

Algorithm 3 Polynomial division by $\mathbb{Z}_m(x)$ in the LCH basis

Require: $A(x) = \sum_{i=0}^{2^{d+m+1}-1} a_i^{\bar{x}} \bar{X}_i(x) \in \mathbb{F}_{2^k}[x]/(x^{2^k} - x)$ of degree $< 2^{d+m+1}$ with $d+m+1 \leq k$, expressed in the LCH basis.

Ensure: $Q(x), R(x) \leftarrow \text{DivLCH}(A(x), d, m)$: $Q(x)$ of degree $< 2^{d+m+1} - 2^m$, and $R(x)$ of degree $< 2^m$ are in the LCH basis and $A(x) = \mathbb{Z}_m(x) \cdot Q(x) + R(x)$.

- 1: $\bar{\mathbf{r}} := [a_i^{\bar{x}} \mid 0 \leq i < 2^m]$, and $\bar{\mathbf{q}} := [a_i^{\bar{x}} \mid 2^m \leq i < 2^{m+d+1}]$.
- 2: $\nu \leftarrow 1_{\mathbb{F}_{2^k}}$
- 3: **for** $s = 0, \dots, d$ **do**
- 4: $\gamma \leftarrow \nu \cdot (p_{2^{m+s}})^{-1}$ $\triangleright$ where $p_{2^{m+s}} = \mathbb{Z}_{m+s}(\beta_{m+s})$ by (1)
- 5: **for** $v = 0, 1, \dots, 2^{d-s} - 1$ **do**
- 6: $t \leftarrow v \cdot 2^{s+1} + 2^s - 1$
- 7: **for** $i = 0, \dots, 2^m - 1$ **do**
- 8: $\bar{\mathbf{q}}[t \cdot 2^m + i] \leftarrow \bar{\mathbf{q}}[t \cdot 2^m + i] \cdot \gamma$
- 9: **for** $u = 0, 1, \dots, 2^s - 2$ **do**
- 10: $w \leftarrow v \cdot 2^{s+1} + u$
- 11: $\bar{\mathbf{q}}[w \cdot 2^m + i] \leftarrow \bar{\mathbf{q}}[w \cdot 2^m + i] + \bar{\mathbf{q}}[t \cdot 2^m + i]$
- 12: **end for**
- 13: **end for**
- 14: **end for**
- 15: $\nu \leftarrow \nu \cdot p_{2^{m+s}}$
- 16: **end for**
- 17: Compute $Q(x), R(x)$ from $\bar{\mathbf{q}}$ and $\bar{\mathbf{r}}$
- 18: **return** $Q(x), R(x)$

Theorem 2. *Algorithm 3 returns $Q(x), R(x)$ in the LCH basis with $A(x) = \mathbb{Z}_m(x) Q(x) + R(x)$, $\deg(R) < 2^m$, and $\deg(Q) < 2^{d+m+1} - 2^m$.*

4.2 Detailed Cost Analysis of Algorithm 3

Algorithm 3 executes the identical mathematical division as Algorithm 2, but transitions the data flow into the LCH basis instead of LCH alternate basis. This allows the expansion polynomial to utilize uniform scalar coefficients, fundamentally restructuring the innermost loops. Let the input polynomial $A(x)$ possess a degree bounded by $n = 2^{m+d+1}$, processed as an array of 2^{d+1} distinct blocks, each containing exactly 2^m field elements.

Proposition 2. *Algorithm 3 performs exactly $2^m(2^{d+1} - 1)$ and $2^m(d \cdot 2^d - 2^d + 1)$ field multiplications and field additions, respectively.*

Proof. The structural advantage of the LCH basis (normalized) is evident in line 8 of Algorithm 3. The scalar multiplication $\bar{\mathbf{q}}[t \cdot 2^m + i] \leftarrow \bar{\mathbf{q}}[t \cdot 2^m + i] \cdot \gamma$ has been generated completely outside of the innermost u -loop (line 9). At any stage $s \in \{0, \dots, d\}$, the v -loop runs 2^{d-s} times and the i -loop runs 2^m times. Summing across all stages yields the exact multiplication count as

$$\sum_{s=0}^d 2^{d-s} \cdot 2^m = 2^m \sum_{s=0}^d 2^{d-s} = 2^m (2^{d+1} - 1).$$

Whereas, the addition count remain same as Algorithm 2. Note that, when $s = 0$, the inner u -loop is empty. Also, the field addition occurs only at line 11. Therefore, the total addition count of the algorithm becomes

$$\sum_{s=1}^d (2^s - 1) \cdot 2^{d-s} \cdot 2^m = 2^m (d \cdot 2^d - 2^d + 1). \quad \square$$

Remark 2. A structural equivalence emerges between Algorithm 3 and Algorithm 2 when the evaluation domain is constructed using the Cantor special basis. As mentioned in Corollary 1, the Cantor special basis forces the scalar constants to $p_i = 1$ for all i . Under this condition, the LCH basis elements $\overline{X}_i(x) = X_i(x)/p_i$ coincide with the alternative basis elements $X_i(x)$. Operationally within Algorithm 3, the scaling factor satisfies $\gamma = 1$ always. Consequently, the scalar multiplication at line 8 becomes an identity operation, effectively reducing the finite field multiplication cost to zero. Furthermore, because the $s = 0$ stage only applies this scaling (with the inner u -loop empty), the entire $s = 0$ stage becomes operationally redundant. By eliminating stage $s = 0$ and the γ multiplications, the data flow of Algorithm 3 collapses perfectly into that of Algorithm 2. Thus, over the Cantor special basis, both algorithms are operationally equivalent, relying exclusively on the identical addition count of $2^m (d \cdot 2^d - 2^d + 1)$. Table 2 compares the exact number of additions and multiplications for both algorithms.

Table 2: Exact arithmetic complexity of polynomial division by $\mathbb{Z}_m(x)$ over $\mathbb{F}_{2^k}$, where the input polynomial $A(x)$ possess a degree bounded by $n = 2^{m+d+1}$ and $\deg \mathbb{Z}_m = 2^m$.

Algorithm	Polynomial Basis	Evaluation Domain Basis	Field Multiplications	Field Additions
Algorithm 2	LCH	General	$2^m(d \cdot 2^d - 2^d + 1)$	$2^m(d \cdot 2^d - 2^d + 1)$
	Alternative	Cantor	0	
Algorithm 3	LCH	General	$2^m(2^{d+1} - 1)$	0
		Cantor	0	

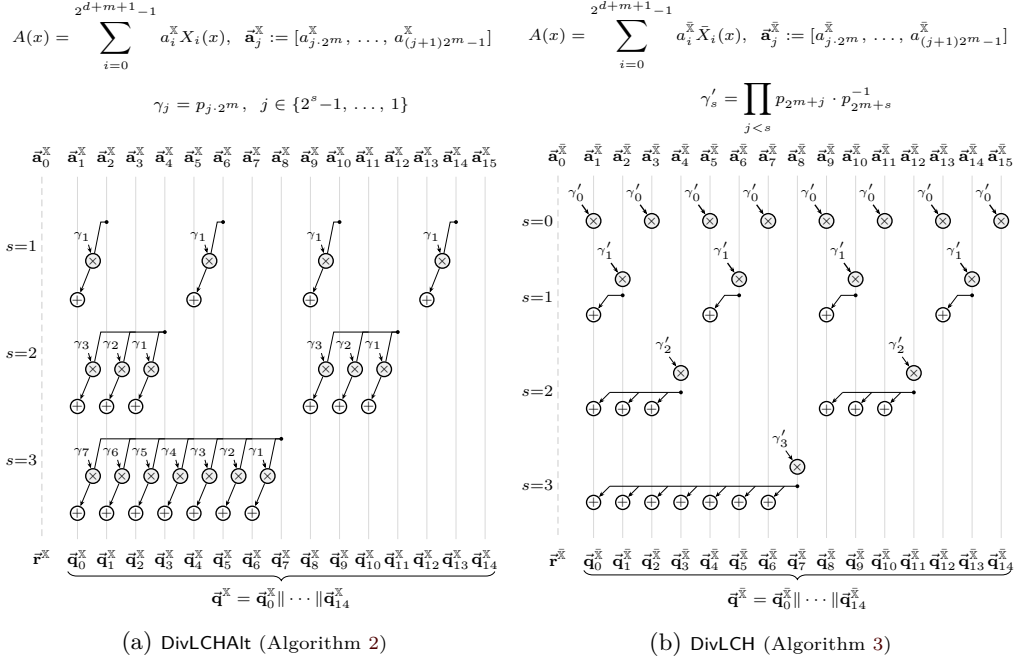


Figure 1: Data flow of Algorithm 2 and Algorithm 3 for $d = 3$ and generic m ; stages proceed top to bottom. Each wire carries a block $\vec{a}_j$ of 2^m coefficients, and every addition ($\oplus$) and multiplication ($\otimes$) operates element-wise on vectors of size 2^m .

5 Polynomial Multiplication Algorithm

Let $h(x) = \sum_{i=0}^{b-1} h_i^{\bar{x}} \bar{X}_i(x)$ with $b \leq 2^m$. Since $\mathbb{Z}_m(x) = p_{2^m} \bar{X}_{2^m}(x)$ has a single nonzero LCH coefficient at index 2^m , and for any $0 \leq i < 2^m$ the indices 2^m and i have disjoint binary representations, the product of basis elements satisfies $\bar{X}_{2^m}(x) \cdot \bar{X}_i(x) = \bar{X}_{2^m+i}(x)$. Therefore, if $h(x) = \sum_{i=0}^{b-1} h_i^{\bar{x}} \bar{X}_i(x)$ with $b \leq 2^m$, we have the product $\mathbb{Z}_m(x) \cdot h(x)$ as

$$\mathbb{Z}_m(x) \cdot h(x) = \sum_{i=0}^{b-1} p_{2^m} h_i^{\bar{x}} \bar{X}_{2^m+i}(x),$$

i.e., the LCH coefficient vector of h is shifted from positions $0, \dots, b-1$ to positions $2^m, \dots, 2^m+b-1$, each scaled by p_{2^m} .

In zero-knowledge proof systems, an oracle polynomial $\hat{f}$ of degree $< 2^m$ is blinded by publishing $\hat{f} + \mathbb{Z}_m \cdot h$ instead of $\hat{f}$, where h is a uniformly random polynomial; since $\mathbb{Z}_m$ vanishes on the constraint domain, the blinding term does not affect the polynomial's values there. Now let $\hat{f}(x) = \sum_{i=0}^{2^m-1} f_i^{\bar{x}} \bar{X}_i(x)$. The coefficients of $\hat{f}$ occupy positions $0, \dots, 2^m-1$ and those of $\mathbb{Z}_m \cdot h$ occupy positions $\geq 2^m$; the two blocks are disjoint. The blinded polynomial $\hat{f} + \mathbb{Z}_m \cdot h$ is thus assembled by concatenating the original LCH coefficients of $\hat{f}$ with the shifted random coefficients of h . Since the scaled coefficients $p_{2^m} h_i^{\bar{x}}$ are uniformly random, the blinding reduces to appending random field elements at LCH positions $\geq 2^m$, eliminating the polynomial multiplication entirely.

One can argue that the blinding could also be performed in the evaluation domain: since $\mathbb{Z}_m$ vanishes on H , the evaluations of $\hat{f} + \mathbb{Z}_m \cdot h$ on H coincide with those of $\hat{f}$, and one could set evaluations on an extended domain $H' \supset H$ to random values to implicitly define the blinding. However, recovering the LCH coefficients of the blinded polynomial would then require a *truncated* IButterfly on the extended domain H' , operating on $2^m + b$ points rather than 2^m . The LCH coefficient approach avoids this overhead, as the IButterfly operates only on H at size 2^m , and the blinding is a zero-cost append of random coefficients. Moreover, the subsequent Butterfly to evaluate the blinded polynomial on the codeword domain L has the same cost in both approaches, since the degree of $\hat{f} + \mathbb{Z}_m \cdot h$ is the same regardless of how the blinding is constructed.

The prover also requires a sumcheck masking polynomial (Phase 3 in Section 3.3) of the form $\hat{m} = \mathbb{Z}_H \cdot h + g$, where h and g are uniformly random polynomials with $\deg(h) < b$ and $\deg(g) < |H|$, subject to the constraint $\sum_{a \in H} g(a) = 0$. Writing $g(x) = \sum_{i=0}^{|H|-1} g_i^{\bar{x}} \bar{X}_i(x)$ in the LCH basis, the constraint admits a simple characterization. Since $\deg(\bar{X}_i) = i$, any polynomial with $g_{|H|-1}^{\bar{x}} = 0$ has degree strictly less than $|H| - 1$ in the monomial basis. By [BCR⁺19, Lemma 5.4], every polynomial of degree $< |H| - 1$ sums to zero over the affine subspace H . Conversely, $\bar{X}_{|H|-1}$ has degree exactly $|H| - 1$ and satisfies $\sum_{a \in H} \bar{X}_{|H|-1}(a) \neq 0$, so $\sum_{a \in H} g(a) = 0$ if and only if $g_{|H|-1}^{\bar{x}} = 0$. The zero-sum condition thus reduces to setting a single LCH coefficient to zero.

6 Auxiliary Prover Optimizations

Beyond the LCH-basis polynomial division and multiplication algorithms, several further optimizations reduce the Aurora prover's workload.

6.1 Sparse R1CS Matrix-Vector Products

The Aurora prover computes the products Az , Bz , Cz (Phase 1) and $M^\top \vec{\alpha}$ for $M \in \{A, B, C\}$ (Phase 2). In the reference implementation, A , B , C are stored as dense $m \times (n+1)$ arrays; each product iterates over all $m(n+1)$ entries with a per-element zero

check. Since the AES R1CS matrices are sparse, with $\|A\| + \|B\| + \|C\| = O(n)$ nonzero entries, we store them in coordinate (COO) format as (r_i, c_i, v_i) triplets and compute each product in $O(\|M\|)$ multiply-accumulate operations, eliminating the zero-check branching overhead entirely.

6.2 Vanishing Polynomial Evaluation over Domains

Several prover stages require the evaluations $\{\mathbb{Z}_m(a)\}_{a \in L}$ of the subspace vanishing polynomial over the codeword domain (FZ Setup, Sumcheck G) or over the codeword domain's basis elements (Rowcheck). The reference implementation constructs $\mathbb{Z}_m$ in the monomial basis via the recurrence, producing $m + 1$ dense coefficients, and evaluates it via an FFT on L or per-point Horner evaluation. In the LCH basis, $\mathbb{Z}_m$ has a single nonzero coefficient at index β_0 , so its evaluations over any affine subspace are computed by a single Butterfly call on a (2^{m+1}) -element input. For the Rowcheck, where $\mathbb{Z}_m$ is evaluated at the $\mathbb{F}_2$ -basis elements $\beta_0, \dots, \beta_{k-1}$ of the codeword domain, the values coincide with the precomputed entries $\mathbb{Z}_m(\beta_i)$ from the basis table, requiring no polynomial evaluation at all.

6.3 Sumcheck Decomposition in the LCH Basis

The sumcheck (Phase 3) requires the coefficient-form decomposition $\hat{f} = \mathbb{Z}_m \cdot \hat{h} + \hat{r}$ with $\deg(\hat{f}) < 2 \cdot 2^m$. The reference implementation constructs the dense monomial-basis vanishing polynomial and applies schoolbook division. When $\hat{f}$ is already represented in LCH coefficients (from the preceding IButterfly), the decomposition reduces to the base case ($d = 0$) of Algorithm 3: the quotient $\hat{h}$ is the upper block of LCH coefficients (positions $2^m, \dots, 2^{m+1}-1$) scaled by $p_{2^m}^{-1}$, and the remainder $\hat{r}$ is the lower block (positions $0, \dots, 2^m-1$). No vanishing-polynomial construction and no iterative long division are required.

6.4 Amortized Domain Exponentiation for the Low-Degree Test

The FRI low-degree test (Phase 5) combines all oracle codewords into a single codeword via a random linear combination. Submaximal-degree oracles must first be degree-shifted: the evaluations of oracle j are multiplied by $\{a^{e_j}\}_{a \in L}$, where e_j is the exponent that raises its degree to the maximum tested degree. The reference implementation computes $\{a^{e_j}\}_{a \in L}$ independently for each oracle j by exponentiating the domain basis elements and calling `all_subset_sums`, recomputing the same basis powers from scratch whenever different oracles share common bit positions in their exponents.

We decompose each exponent into its binary representation $e_j = \sum_i 2^{b_i}$ and precompute, for each needed bit position b_i , the table $T_{b_i} = \{a^{2^{b_i}}\}_{a \in L}$ via iterated squaring of the domain basis followed by a single `all_subset_sums` call. For power-of-two exponents, T_{b_i} is used directly; for general exponents, the relevant tables are multiplied element-wise. Results are cached by exponent so that multiple oracles sharing the same degree shift incur no redundant computation.

7 Performance Evaluation

We evaluate the signing latency of Preon on an AMD Ryzen 9 9950X (PCLMULQDQ-capable, 5.7 GHz boost) with 64 GB DDR5 RAM, running Debian 12 (kernel 6.12.12), compiled with GCC 14.2. We report the median of 100 timed iterations preceded by 10 warmup iterations for Preon-128A (NIST Level I, aggressive security assumptions) and Preon-256C (NIST Level V, conservative security assumptions); all measurements correspond to the end-to-end `crypto_sign` wall-clock time.

Experimental configurations. We compare three configurations: *Baseline*¹ (unmodified reference implementation); **CLMUL** (carry-less multiply PCLMULQDQ intrinsics for field arithmetic, no algorithmic changes); **NativeLCH** (CLMUL combined with the LCH-basis algorithms of Sections 4–6). Table 1 summarises the end-to-end results.

CLMUL alone yields 1.6–1.9 $\times$ by accelerating individual field operations, but cannot eliminate the $O(n \log^2 n)$ Gao–Mateer FFT. NativeLCH provides an additional 2.6–3.5 $\times$ on top of CLMUL, for a combined **5.0–5.8 $\times$** speedup over the Baseline.

7.1 Prover Pipeline Breakdown

Figure 2 shows the sequential stages of the Aurora/Preon prover, organized by IOP phase. Tables 3 and 4 break down the wall time into non-overlapping pipeline stages for both parameter sets. The rows labeled *Other* aggregate minor bookkeeping (R1CS witness construction, virtual-oracle initialization, query-position derivation, and proof serialization); *Framework Overhead* is the residual `crypto_sign` time not covered by any profiled region. This residual drops sharply in NativeLCH, as dense R1CS matrix construction is replaced by a sparse variant and absorbed into the *Witness & Constraint Setup*.

The polynomial transform (Gao–Mateer FFT in Baseline/CLMUL; LCH Butterfly in NativeLCH) is a cross-cutting primitive embedded within multiple stages, not a standalone phase; Table 5 isolates its aggregate cost.

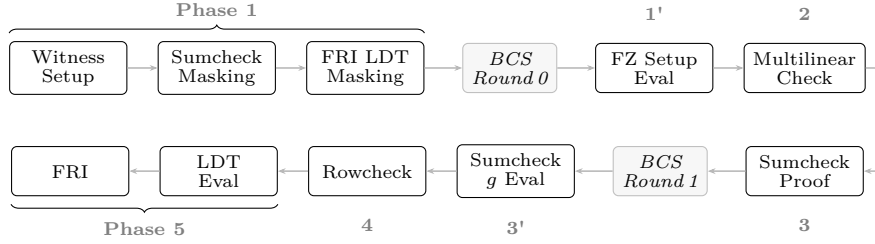


Figure 2: Aurora/Preon prover pipeline. Flow proceeds left to right on the top row and back on the bottom row. Gray “BCS” nodes are Merkle-commit + Fiat–Shamir rounds separating IOP phases.

7.2 Polynomial Transform Cost

The Gao–Mateer FFT consumes over 40% of Baseline signing time (Table 5). CLMUL reduces the absolute transform time by 1.3–1.5 $\times$, but because non-transform stages shrink less, the FFT’s relative share *rises* to 53–56%. NativeLCH replaces the $O(n \log^2 n)$ FFT with the $O(n \log n)$ LCH butterfly, collapsing the transform to 13–17% of total time and yielding a **12.6–17.9 $\times$** speedup that accounts for roughly half of the end-to-end gain. The advantage grows with security level, as the butterfly’s share drops from 17.2% (Preon-128A) to 13.0% (Preon-256C), while the Baseline share remains $\sim 40\%$.

7.3 Discussion

Phases 1–3: algorithmic gains. Oracle Construction, Lincheck, and Sumcheck collectively account for 46–48% of Baseline time but collapse to 11–15% in NativeLCH. In *Witness & Constraint Setup*, the butterfly replaces the Gao–Mateer FFT for all interpolation and evaluation, LCH-basis division replaces schoolbook division for $\hat{f}_w$, direct ZK blinding via LCH coefficient append eliminates the vanishing-polynomial multiplication, and sparse COO products replace dense R1CS iteration for Az , Bz , Cz . The *Sumcheck and FRI LDT*

¹The reference implementation contained memory leaks that inflated timing measurements; these were resolved in all benchmark configurations to ensure accurate and fair comparison.

Table 3: Per-stage wall time (ms) for Preon-128A, organized by Aurora IOP phase. Percentages are of the end-to-end `crypto_sign` time.

Phase	Stage	Baseline	CLMUL	NativeLCH
1: Oracle Construction	Witness & Constraint Setup	1,913 (16.1%)	1,285 (20.6%)	138 (5.8%)
	Sumcheck Masking (ZK)	538 (4.5%)	291 (4.7%)	36 (1.5%)
	FRI LDT Masking (ZK)	569 (4.8%)	292 (4.7%)	38 (1.6%)
	Merkle Round 0	448 (3.8%)	464 (7.5%)	444 (18.6%)
	FZ Setup Evaluation	546 (4.6%)	584 (9.4%)	77 (3.2%)
2: Lincheck	Multilinear Check	1,838 (15.4%)	679 (10.9%)	104 (4.3%)
3: Sumcheck	Sumcheck Proof	747 (6.3%)	590 (9.5%)	48 (2.0%)
	Merkle Round 1	361 (3.0%)	374 (6.0%)	357 (14.9%)
	Sumcheck G	285 (2.4%)	296 (4.8%)	42 (1.8%)
4: Rowcheck	Rowcheck	207 (1.7%)	20 (0.3%)	17 (0.7%)
5: Low-Degree Test	LDT Challenge Eval	2,967 (24.9%)	660 (10.6%)	476 (19.9%)
	FRI LDT	1,448 (12.2%)	646 (10.4%)	612 (25.6%)
—	Other	1 (<0.1%)	1 (<0.1%)	1 (<0.1%)
	Framework Overhead	42 (0.4%)	44 (0.7%)	1 (<0.1%)
Total		11,910 (100%)	6,226 (100%)	2,391 (100%)

Masking stages similarly benefit from the butterfly and direct LCH coefficient sampling. In *FZ Setup* and *Sumcheck G*, the single-coefficient LCH representation of $\mathbb{Z}_H$ reduces vanishing-polynomial evaluation to a single butterfly call. The *Multilinear Check* gains from the butterfly and sparse COO products for $M^\top \vec{\alpha}$. The *Sumcheck Proof* in Preon-256C is particularly striking: CLMUL provides no improvement (its share grows from 10.8% to 19.6%), yet NativeLCH collapses it to 1.9% by replacing the schoolbook division with a base-case LCH-basis split and running the IButterfly on a minimal subspace of size $2t$ rather than $|L|$.

Residual bottleneck: Phase 5 and Merkle hashing. Phase 5 (LDT/FRI) and Merkle hashing (Rounds 0 + 1) are largely unaffected by the LCH restructuring and become the dominant residual cost. *LDT Challenge Eval* benefits from amortized domain exponentiation via shared bit-level tables and CLMUL-accelerated element-wise multiplications, but the *FRI folding loop* is unchanged in the current work. Merkle hashing is SHA-based and entirely invariant across configurations. These hash-dominated and FRI-dominated stages define the performance floor for further optimization.

8 Conclusions and Future Work

We presented a divide-and-conquer algorithm for polynomial division by subspace vanishing polynomials that operates directly in the LCH polynomial basis, for arbitrary $\mathbb{F}_2$ -basis elements. This algorithm let us eliminate the basis-conversion stage from every additive FFT and IFFT call, each polynomial transform reduces to the $O(n \log n)$ butterfly evaluation alone, removing the $O(n(\log n)^2)$ -addition and $O(n \log n)$ -multiplication conversion overhead that dominates in practice. The LCH basis further simplifies the multiplication of a zero-knowledge hiding polynomial by a vanishing polynomial reduces to appending random coefficients. Together, these primitives enable the entire Aurora prover pipeline to operate natively in the LCH coefficient representation, with no round-trip to the monomial basis.

Integrating these algorithms into the Preon post-quantum signature scheme, a NIST PQC Round-1 candidate built on the Aurora IOP, yields end-to-end signing speedups of $5.0\times$ for Preon-128A and $5.8\times$ for Preon-256C over the unmodified reference implementa-

Table 4: Per-stage wall time (ms) for Preon-256C, organized by Aurora IOP phase. Percentages are of the end-to-end `crypto_sign` time.

Phase	Stage	Baseline	CLMUL	NativeLCH
1: Oracle Construction	Witness & Constraint Setup	19,672 (16.9%)	15,008 (21.0%)	880 (4.3%)
	Sumcheck Masking (ZK)	4,694 (4.0%)	3,118 (4.4%)	234 (1.2%)
	FRI LDT Masking (ZK)	4,891 (4.2%)	3,120 (4.4%)	248 (1.2%)
	Merkle Round 0	3,695 (3.2%)	3,714 (5.2%)	3,855 (19.0%)
	FZ Setup Evaluation	4,977 (4.3%)	6,271 (8.8%)	532 (2.6%)
2: Lincheck	Multilinear Check	18,362 (15.7%)	7,540 (10.6%)	857 (4.2%)
3: Sumcheck	Sumcheck Proof	12,559 (10.8%)	13,985 (19.6%)	382 (1.9%)
	Merkle Round 1	2,672 (2.3%)	2,686 (3.8%)	2,786 (13.8%)
	Sumcheck G	2,589 (2.2%)	3,174 (4.4%)	304 (1.5%)
4: Rowcheck	Rowcheck	2,054 (1.8%)	177 (0.2%)	167 (0.8%)
5: Low-Degree Test	LDT Challenge Eval	24,791 (21.2%)	6,928 (9.7%)	5,368 (26.5%)
	FRI LDT	14,537 (12.5%)	4,419 (6.2%)	4,614 (22.8%)
—	Other	4 (<0.1%)	4 (<0.1%)	6 (<0.1%)
	Framework Overhead	1,206 (1.0%)	1,208 (1.7%)	5 (<0.1%)
Total		116,703 (100%)	71,352 (100%)	20,238 (100%)

Table 5: Aggregate polynomial-transform time (FFT/Butterfly) across all prover stages.

Configuration	Preon-128A		Preon-256C	
	Time (ms)	% of total	Time (ms)	% of total
Baseline (FFT + IFFT)	5,162	43.3%	47,117	40.4%
CLMUL (FFT + IFFT)	3,469	55.7%	37,289	52.3%
NativeLCH (Butterfly + IBfly)	411	17.2%	2,630	13.0%

tion, with the polynomial transform alone 12.6–17.9× faster. Since all changes are confined to the polynomial arithmetic layer, they are directly applicable to any univariate-polynomial IOP prover over binary extension fields, including STARK, Fractal, and Polaris.

Two directions for future work follow naturally. First, because eliminating the basis-conversion phase leaves a butterfly structure that is identical to the Cooley–Tukey FFT [CT65], the parallelization techniques that have yielded order-of-magnitude speedups for multiplicative-FFT zkSNARK provers on GPUs [LWY⁺23, MXS⁺23] become directly applicable to the additive setting, removing the global synchronization barrier and irregular parallelism patterns that made the two-phase additive FFT difficult to parallelize in prior work [BSHST16]. Second, our implementation retains the evaluation-domain basis of the reference code to preserve bit-identical proof transcripts; however, as shown in Corollary 1, switching to the Cantor special basis would reduce all field multiplications in both the division algorithm and the butterfly evaluation to zero, leaving only additions.

A Proofs

A.1 Proof of Correctness for Algorithm 1

Proof. Fix $m \geq 0$. We proceed by induction on the recursion depth $d \geq 0$. Consider a polynomial

$$A(x) = \sum_{i=0}^{2^{d+m+1}-1} a_i^{\mathbb{X}} X_i(x) \in \mathbb{F}_{2^k}[x]/(x^{2^k} - x),$$

with $\deg(A) < 2^{d+m+1}$, expressed in the LCH alternative basis. We must show that the Algorithm 1 returns polynomials $Q(x)$ and $R(x)$ such that

$$A(x) = \mathbb{Z}_m(x) \cdot Q(x) + R(x), \quad \deg(R) < 2^m, \quad \deg(Q) < 2^{d+m+1} - 2^m.$$

Base case $d = 0$. When $d = 0$, the algorithm sets

$$R(x) = \sum_{i=0}^{2^m-1} a_i^{\mathbb{X}} X_i(x), \quad Q(x) = \sum_{i=0}^{2^m-1} a_{i+2^m}^{\mathbb{X}} X_i(x),$$

and returns $(Q(x), R(x))$. By Lemma 1, this is exactly the quotient and remainder of dividing $A(x)$ by $\mathbb{Z}_m(x)$, and we have $A(x) = \mathbb{Z}_m(x) Q(x) + R(x)$ with $\deg(R) < 2^m$ and $\deg(Q) < 2^{m+1} - 2^m = 2^m$.

Induction step. Assume that the claim holds for some integer $d-1 \geq 0$. We will show that it also holds for d . The algorithm decomposes $A(x)$ into two halves:

$$A_0(x) = \sum_{i=0}^{2^{m+d}-1} a_i^{\mathbb{X}} X_i(x), \quad \text{and} \quad A_1(x) = \sum_{i=0}^{2^{m+d}-1} a_{i+2^{m+d}}^{\mathbb{X}} X_i(x).$$

By the structure of the LCH alternative basis, this decomposition satisfies

$$A(x) = A_0(x) + A_1(x) X_{2^{m+d}}(x).$$

By the induction hypothesis, the two recursive calls correctly divide $A_0(x)$ and $A_1(x)$ by $\mathbb{Z}_m(x)$, yielding:

$$A_0(x) = \mathbb{Z}_m(x) Q_0(x) + R_0(x), \quad \text{where} \quad \deg(R_0) < 2^m, \quad \deg(Q_0) < 2^{d+m} - 2^m$$

$$A_1(x) = \mathbb{Z}_m(x) Q_1(x) + R_1(x), \quad \text{where} \quad \deg(R_1) < 2^m, \quad \deg(Q_1) < 2^{d+m} - 2^m.$$

The algorithm sets $R(x) = R_0(x)$, so the required bound $\deg(R) < 2^m$ is immediate.

Next, define the expansion polynomial as $D_{d,m}(x) := \sum_{i=0}^{2^d-1} p_{2^m(2^d-i-1)} X_{2^m i}(x)$, where p_j are defined in (1). Since $W_m \subseteq W_{m+d}$, the vanishing polynomial $\mathbb{Z}_m(x)$ divides $\mathbb{Z}_{m+d}(x) = X_{2^{m+d}}(x)$. The explicit quotient is determined iteratively by the recurrence relation $\mathbb{Z}_{j+1}(x) = \mathbb{Z}_j(x)^2 + p_{2^j} \mathbb{Z}_j(x)$ for $j = m, \dots, m+d-1$ and in the LCH alternative basis the quotient is precisely $D_{d,m}(x)$. Thus,

$$X_{2^{m+d}}(x) = \mathbb{Z}_m(x) D_{d,m}(x). \quad (4)$$

For the quotient, the algorithm defines

$$Q(x) = Q_0(x) + R_1(x) D_{d,m}(x) + Q_1(x) X_{2^{m+d}}(x). \quad (5)$$

We have $\deg(D_{d,m}) = 2^{m+d} - 2^m$ because $\deg(X_{2^{m+d}}) = 2^{m+d}$ and $\deg(\mathbb{Z}_m) = 2^m$. Therefore, we have

$$\deg(R_1 D_{d,m}) < (2^m - 1) + (2^{m+d} - 2^m) = 2^{m+d} - 1,$$

$$\deg(Q_1 X_{2^{m+d}}) < (2^{m+d} - 2^m) + 2^{m+d} = 2^{m+d+1} - 2^m,$$

which implies that $\deg(Q) < 2^{d+m+1} - 2^m$.

It remains to verify the division identity. Using the decompositions of A_0 and A_1 , we obtain

$$\begin{aligned} A(x) &= A_0(x) + A_1(x) X_{2^{m+d}}(x) \\ &= (\mathbb{Z}_m(x) Q_0(x) + R_0(x)) + (\mathbb{Z}_m(x) Q_1(x) + R_1(x)) X_{2^{m+d}}(x) \\ &= R_0(x) + \mathbb{Z}_m(x) Q_0(x) + \mathbb{Z}_m(x) Q_1(x) X_{2^{m+d}}(x) + R_1(x) X_{2^{m+d}}(x) \\ &= R(x) + \mathbb{Z}_m(x) Q_0(x) + \mathbb{Z}_m(x) Q_1(x) X_{2^{m+d}}(x) + R_1(x) \mathbb{Z}_m(x) D_{d,m}(x) \quad [\text{by (4)}] \\ &= R(x) + \mathbb{Z}_m(x) [Q_0(x) + Q_1(x) X_{2^{m+d}}(x) + R_1(x) D_{d,m}(x)] \\ &= R(x) + \mathbb{Z}_m(x) Q(x) \quad [\text{by (5)}]. \end{aligned}$$

Thus, for every $d \geq 0$, Algorithm 1 returns polynomials $Q(x)$ and $R(x)$ such that $A(x) = \mathbb{Z}_m(x)Q(x) + R(x)$ with $\deg(R) < 2^m$ and $\deg(Q) < 2^{d+m+1} - 2^m$. $\square$

A.2 Equivalence of Algorithms 1 and 2.

Proof. We prove the claim by induction on the outer-loop stage $s = 1, \dots, d$ of Algorithm 2. Let $\vec{q}^{(s)}$ denote the state of $\vec{q}$ after stage s , and $\vec{q}^{(0)}$ its initial state.

Invariant $\mathcal{I}(s)$. After stage s , each block group is indexed by $v \in \{0, \dots, 2^{d-s} - 1\}$ stores, in $\vec{q}^{(s)}$, exactly the quotient coefficients obtained by applying s recursive merge levels of Algorithm 1 to the corresponding recursion subtree.

Base case ($s = 0$). Before the loop, $\vec{q}^{(0)}$ is the direct coefficient layout used by the recursive algorithm before any merge step; therefore $\mathcal{I}(0)$ holds.

Induction step. Assume $\mathcal{I}(s-1)$ for some $1 \leq s \leq d$. Fix a group v . In the recursive combine step at depth s , one adds the term $R_1(x)D_{s,m}(x)$ to $Q_0(x)$, where

$$D_{s,m}(x) = \sum_{u=0}^{2^s-1} p_{2^m(2^s-u-1)} X_{2^m u}(x).$$

In Algorithm 2, for each $u = 0, \dots, 2^s - 2$ and local coefficient index i , the update

$$\vec{q}[w2^m + i] \leftarrow \vec{q}[w2^m + i] + \vec{q}[t2^m + i] \cdot p_{2^m(2^s-u-1)},$$

with $t = v2^{s+1} + 2^s - 1$ and $w = v2^{s+1} + u$ adds to target block w exactly the contribution of the source block t scaled by the corresponding coefficient of $D_{s,m}(x)$. Thus the iterative updates reproduce the same linear combination as the recursive merge formula

$$Q = Q_0 + R_1 D_{s,m} + Q_1 X_{2^{m+s}}.$$

Hence, each group after the stage s matches the recursive result after one more merge level, so $\mathcal{I}(s)$ holds.

By induction, $\mathcal{I}(d)$ holds. Therefore, after the last stage, the vector $\vec{q}$ equals the quotient produced by Algorithm 1. Also, $\vec{r}$ is never modified, so it equals the same remainder block used in the recursive algorithm. Hence, both algorithms return identical $(Q(x), R(x))$. $\square$

A.3 Proof of correctness of Algorithm 3.

We first establish two identities specific to the LCH basis, then prove the algorithm correct by induction.

Lemma 3 (LCH-basis quotient by $\mathbb{Z}_m$). *Let $A(x) = \sum_{i=0}^{2^{m+1}-1} a_i^{\bar{x}} \bar{X}_i(x)$ be a polynomial in LCH basis with $\deg(A) < 2^{m+1}$. Then*

$$A(x) = \mathbb{Z}_m(x) \cdot Q(x) + R(x),$$

$$\text{where } R(x) = \sum_{i=0}^{2^m-1} a_i^{\bar{x}} \bar{X}_i(x) \text{ and } Q(x) = \frac{1}{p_{2^m}} \sum_{i=0}^{2^m-1} a_{i+2^m}^{\bar{x}} \bar{X}_i(x).$$

Proof. For $0 \leq i < 2^m$, the indices i and 2^m have disjoint bit sets, so $\bar{X}_{2^m+i} = \bar{X}_{2^m} \cdot \bar{X}_i$. Since $\bar{X}_{2^m} = X_{2^m}/p_{2^m} = \mathbb{Z}_m(x)/p_{2^m}$, we obtain

$$\sum_{i=0}^{2^m-1} a_{i+2^m}^{\bar{x}} \bar{X}_{2^m+i} = \frac{\mathbb{Z}_m(x)}{p_{2^m}} \sum_{i=0}^{2^m-1} a_{i+2^m}^{\bar{x}} \bar{X}_i(x) = \mathbb{Z}_m(x) Q(x). \quad \square$$

Lemma 4 (Uniform LCH coefficients of $D_{s,m}$). *For $D_{s,m}(x) = \sum_{j=0}^{2^s-1} p_{2^m(2^s-j-1)} X_{2^m j}(x)$, we have*

$$D_{s,m}(x) = p_{2^m(2^s-1)} \sum_{j=0}^{2^s-1} \bar{X}_{2^m j}(x). \quad (6)$$

That is, all LCH-basis coefficients of $D_{s,m}$ at the block level are the same constant $p_{2^m(2^s-1)}$.

Proof. Rewriting $X_{2^m j} = p_{2^m j} \bar{X}_{2^m j}$ gives $D_{s,m} = \sum_j p_{2^m(2^s-j-1)} p_{2^m j} \bar{X}_{2^m j}$. The indices $2^m(2^s-j-1)$ and $2^m j$ occupy positions $m, \dots, m+s-1$ with complementary bits (since $(2^s-j-1) + j = 2^s-1$), so their bit sets are disjoint and $p_{2^m(2^s-j-1)} \cdot p_{2^m j} = p_{2^m(2^s-1)}$ for every j . The constant factors out, giving (6). $\square$

A.3.1 Proof of Lemma 2

Proof. The proof mirrors the induction on d used for Algorithm 1, but works entirely in the LCH basis.

Stage $s = 0$ (base-case divisions). When $s = 0$, the inner u -loop runs over $u = 0, \dots, 2^0 - 2 = -1$ and is therefore empty. The algorithm scales source blocks B_{2^v} (for $v = 0, \dots, 2^d - 1$) by $\gamma = 1/p_{2^m}$. By Lemma 3, this is exactly the factor needed to convert the raw LCH coefficients of each leaf's upper half into the LCH coefficients of the corresponding quotient. After this stage, every even-indexed block holds the correct leaf-level quotient, and every odd-indexed block holds the corresponding remainder R_1 (unchanged, since the remainder requires no scaling). The accumulator becomes $\nu = p_{2^m}$.

Stages $s = 1, \dots, d$ (recursive merges). At each stage $s \geq 1$, the recursive combine formula in the LCH basis is

$$Q = Q_0 + \frac{D_{s,m}}{p_{2^{m+s}}} R_1 + \bar{X}_{2^{m+s}} Q_1,$$

where $\bar{X}_{2^{m+s}} = X_{2^{m+s}}/p_{2^{m+s}}$ and $X_{2^{m+s}} = \mathbb{Z}_m(x) D_{s,m}(x)$ by (4). By Lemma 4, the factor multiplying R_1 is

$$\frac{D_{s,m}(x)}{p_{2^{m+s}}} = \frac{p_{2^m(2^s-1)}}{p_{2^{m+s}}} \sum_{j=0}^{2^s-1} \bar{X}_{2^m j}(x),$$

so its contribution to every target block is the *same* scalar $\mu_s := p_{2^m(2^s-1)}/p_{2^{m+s}}$ times the source block. Since $p_{2^m(2^s-1)} = \prod_{j=m}^{m+s-1} p_{2^j}$, we have $\mu_s = \prod_{j=0}^{s-1} p_{2^{m+j}}/p_{2^{m+s}}$, which equals $\gamma = \nu/p_{2^{m+s}}$ because $\nu = \prod_{j=0}^{s-1} p_{2^{m+j}}$ at the start of stage s . The algorithm therefore scales the source block by $\gamma = \mu_s$ and adds the result to each target block, which is exactly the recursive merge. The Q_1 term occupies the high blocks and is already in place. After the stage, ν is updated to $\nu \cdot p_{2^{m+s}} = \prod_{j=0}^s p_{2^{m+j}}$.

Remainder. The vector $\vec{r}$ is never modified. For indices $0 \leq i < 2^m$, the LCH basis element $\bar{X}_i$ does not involve $\mathbb{Z}_m(x)$ (bit m is not set in i), so $\sum_{i < 2^m} a_i^{\vec{r}} \bar{X}_i$ is the part of $A(x)$ not divisible by $\mathbb{Z}_m(x)$ and is thus the correct remainder.

By induction, after all $d+1$ stages the quotient and remainder satisfy $A(x) = \mathbb{Z}_m(x) Q(x) + R(x)$ with the stated degree bounds. $\square$

References

- [AHIV17] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkatasubramanian. Liger: Lightweight sublinear arguments without a trusted setup. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 2087–2104. ACM Press, October / November 2017.

- [ark] arkworks contributors. arkworks zkSNARK ecosystem. <https://arkworks.rs>. Source code available at <https://github.com/arkworks-rs>. Accessed: 2026-04-10.
- [BC14] Daniel J. Bernstein and Tung Chou. Faster binary-field multiplication and faster binary-field MACs. In Antoine Joux and Amr M. Youssef, editors, *Selected Areas in Cryptography - SAC 2014 - 21st International Conference, Montreal, QC, Canada, August 14-15, 2014, Revised Selected Papers*, volume 8781 of *Lecture Notes in Computer Science*, pages 92–111. Springer, 2014.
- [BCCT12] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, ITCS '12, page 326–349, New York, NY, USA, 2012. Association for Computing Machinery.
- [BCR⁺19] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for R1CS. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part I*, volume 11476 of *Lecture Notes in Computer Science*, pages 103–128. Springer, 2019.
- [BdSGK⁺23] Carsten Baum, Cyprien Delpéch de Saint Guilhem, Michael Klooß, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023*, pages 581–615, Cham, 2023. Springer Nature Switzerland.
- [BGoc] Sean Bowe, Jack Grigg, and other contributors. bellman: a Rust library for zk-SNARKs. <https://github.com/zkcrypto/bellman>. Full list of contributors available at <https://github.com/zkcrypto/bellman/graphs/contributors>. Accessed: 2026-04-10.
- [BGPoc] Gautam Botrel, Gus Gutoski, Thomas Piellard, and other contributors. gnark: a fast zk-SNARK library in Go. <https://github.com/Consensys/gnark>. Full list of contributors available at <https://github.com/Consensys/gnark/graphs/contributors>. Accessed: 2026-04-10.
- [BLG⁺] Sean Bowe, Ying Tong Lai, Jack Grigg, Daira Hopwood, and other contributors. halo2: a Rust implementation of the Halo2 zero-knowledge proving system. <https://github.com/zcash/halo2>. Full list of contributors available at <https://github.com/zcash/halo2/graphs/contributors>. Accessed: 2026-04-10.
- [BPoc] Jordi Baylina, Xavi Pinsach, and other contributors. snarkjs: a JavaScript and WebAssembly implementation of zkSNARK and PLONK schemes. <https://github.com/iden3/snarkjs>. Full list of contributors available at <https://github.com/iden3/snarkjs/graphs/contributors>. Accessed: 2026-04-10.
- [BSBHR18a] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon interactive oracle proofs of proximity. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *45th International Colloquium on Automata, Languages, and Programming*

- (ICALP 2018), volume 107 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 14:1–14:17, Dagstuhl, Germany, 2018. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [BSBHR18b] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Paper 2018/046, 2018.
- [BSCG⁺] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Shaul Kfir, Eran Tromer, Madars Virza, Howard Wu, and other contributors. libsnark: a C++ library for zkSNARKs. <https://github.com/scipr-lab/libsnark>. Full list of contributors available at <https://github.com/scipr-lab/libsnark/blob/master/AUTHORS>. Accessed: 2026-04-10.
- [BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In *Proceedings, Part II, of the 14th International Conference on Theory of Cryptography - Volume 9986*, page 31–60, Berlin, Heidelberg, 2016. Springer-Verlag.
- [BSG26] Mohammadtaghi Badakhshan, Susanta Samanta, and Guang Gong. Accelerating post-quantum secure zkSNARKs by optimizing additive FFT. In Christina Boura, Atefeh Mashatan, and Ali Miri, editors, *Selected Areas in Cryptography – SAC 2025*, pages 339–368, Cham, 2026. Springer Nature Switzerland.
- [BSHST16] Eli Ben-Sasson, Matan Hamilis, Mark Silberstein, and Eran Tromer. Fast multiplication in binary fields on GPUs via register cache. In *Proceedings of the 30th International Conference on Supercomputing (ICS)*, 2016.
- [BWL_{oc}] William Borgeaud, Nicholas Ward, Daniel Lubarov, and other contributors. Plonky2: a fast recursive SNARK based on PLONK and FRI. <https://github.com/0xPolygonZero/plonky2>. Full list of contributors available at <https://github.com/0xPolygonZero/plonky2/graphs/contributors>. Accessed: 2026-04-10.
- [BZG⁺26] Ryan Babbush, Adam Zalcman, Craig Gidney, Michael Broughton, Tanuj Khatkar, Hartmut Neven, Thiago Bergamaschi, Justin Drake, and Dan Boneh. Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations. *arXiv:2603.28846*, 2026. <https://arxiv.org/abs/2603.28846>.
- [Can89] David G. Cantor. On arithmetical algorithms over finite fields. *J. Comb. Theory A*, 50(2):285–300, 1989.
- [CCC⁺23] Ming-Shing Chen, Yu-Shian Chen, Chen-Mou Cheng, Shiuan Fu, Wei-Chih Hong, Jen-Hsuan Hsiang, Sheng-Te Hu, Po-Chun Kuo, Wei-Bin Lee, Feng-Hao Liu, et al. Preon: zk-SNARK based Signature Scheme. *Technical report. NIST*, 2023.
- [CCK⁺17] Ming-Shing Chen, Chen-Mou Cheng, Po-Chun Kuo, Wen-Ding Li, and Bo-Yin Yang. Faster multiplication for long binary polynomials. *CoRR*, abs/1708.09746, 2017.
- [CJL⁺24] Kostas Kryptos Chalkias, Charanjit Jutla, Jonas Lindstrom, Varun Madathil, and Arnab Roy. Improved Polynomial Division in Cryptography. Cryptology ePrint Archive, Paper 2024/1279, 2024.

- [COS20] Alessandro Chiesa, Dev Ojha, and Nicholas Spooner. Fractal: Post-quantum and Transparent Recursive Proofs from Holography. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020*, pages 769–793, Cham, 2020. Springer International Publishing.
- [CT65] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.
- [DP25] Benjamin E. Diamond and Jim Posen. Succinct arguments over towers of binary fields. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025*, pages 93–122, Cham, 2025. Springer Nature Switzerland.
- [FG22] Shihui Fu and Guang Gong. Polaris: Transparent Succinct Zero-Knowledge Arguments for R1CS with Efficient Verifier. *Proceedings on Privacy Enhancing Technologies*, 2022.
- [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology — CRYPTO’ 86*, pages 186–194, Berlin, Heidelberg, 1987. Springer Berlin Heidelberg.
- [FS24] Matteo Frigo and Abhi Shelat. Anonymous credentials from ECDSA. Cryptology ePrint Archive, Paper 2024/2010, 2024.
- [GM10] Shuhong Gao and Todd D. Mateer. Additive fast Fourier transforms over finite fields. *IEEE Trans. Inf. Theory*, 56(12):6265–6272, 2010.
- [Gro16] Jens Groth. On the Size of Pairing-Based Non-interactive Arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology – EUROCRYPT 2016*, pages 305–326, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg.
- [GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. Cryptology ePrint Archive, Paper 2019/953, 2019.
- [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *Advances in Cryptology - ASIACRYPT 2010*, pages 177–194, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.
- [LAH16] Sian-Jheng Lin, Tareq Y. Al-Naffouri, and Yunghsiang S. Han. FFT algorithm for binary extension finite fields and its application to Reed-Solomon codes. *IEEE Trans. Inf. Theory*, 62(10):5343–5358, 2016.
- [LAHC16] Sian-Jheng Lin, Tareq Y. Al-Naffouri, Yunghsiang Sam Han, and Wei-Ho Chung. Novel Polynomial Basis with Fast Fourier Transform and Its Application to Reed-Solomon Erasure codes. *IEEE Transactions on Information Theory*, 62(11):6284–6299, 2016.
- [LCH14] Sian-Jheng Lin, Wei-Ho Chung, and Yunghsiang S. Han. Novel polynomial basis and its application to Reed-Solomon erasure codes. In *55th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2014, Philadelphia, PA, USA, October 18-21, 2014*, pages 316–325. IEEE Computer Society, 2014.

- [LWY⁺23] Tao Lu, Chengkun Wei, Ruijing Yu, Chaochao Chen, Wenjing Fang, Lei Wang, Zeke Wang, and Wenzhi Chen. cuZK: Accelerating zero-knowledge proof with a faster parallel multi-scalar multiplication algorithm on GPUs. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(3):194–220, 2023.
- [Mon87] Peter L. Montgomery. Speeding the Pollard and Elliptic Curve Methods of Factorization. *Mathematics of Computation*, 48:243–264, 1987.
- [MXS⁺23] Weiliang Ma, Qian Xiong, Xuanhua Shi, Xiaosong Ma, Hai Jin, Haozhao Kuang, Mingyu Gao, Ye Zhang, Haichen Shen, and Weifang Hu. GZKP: A GPU accelerated zero-knowledge proof system. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 340–353. ACM, 2023.
- [NIST23] National Institute of Standards and Technology. Post-Quantum Cryptography: Standardization Process. <https://csrc.nist.gov/projects/pqc-dig-sig/round-1-additional-signatures>, 2023. Accessed: 2025-01-10.
- [RS60] Irving Stoy Reed and Gustave Solomon. Polynomial codes over certain finite fields. *Journal of the Society for Industrial and Applied Mathematics*, 8(2):300–304, 1960.
- [Sch80] J. T. Schwartz. Fast Probabilistic Algorithms for Verification of Polynomial Identities. *J. ACM*, 27(4):701–717, October 1980.
- [Set20] Srinath Setty. Spartan: Efficient and General-Purpose zkSNARKs Without Trusted Setup. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020*, pages 704–737, Cham, 2020. Springer International Publishing.
- [Zip79] Richard Zippel. Probabilistic algorithms for sparse polynomials. In Edward W. Ng, editor, *Symbolic and Algebraic Computation*, pages 216–226, Berlin, Heidelberg, 1979. Springer Berlin Heidelberg.